

Decorator

Design Pattern

Situação: Notificar



Solução Clássica

```
public class Notifier {  
    private final String username;  
    protected final DatabaseService databaseService;  
  
    public Notifier(String username) {  
        this.username = username;  
        databaseService = new DatabaseService();  
    }  
  
    public void send(String msg) {  
        String mail = databaseService.getMailFromUsername(username);  
        System.out.println("Sending " + msg + " by Mail to " + mail);  
    }  
  
    public String getUsername() {  
        return username;  
    }  
}
```

```
public class DatabaseService {  
    public String getMailFromUsername(String username) {  
        return username + "@Mail";  
    }  
}
```

Daí alguém pergunta: e se...?

```
public class Notifier {  
    private final String username;  
    protected final DatabaseService databaseService;  
  
    public Notifier(String username) {  
        this.username = username;  
        databaseService = new DatabaseService();  
    }  
  
    public void send(String msg) {  
        String mail = databaseService.getMailFromUsername(username);  
        System.out.println("Sending " + msg + " by Mail to " + mail);  
    }  
  
    public String getUsername() {  
        return username;  
    }  
}
```

```
public class DatabaseService {  
    public String getMailFromUsername(String username) {  
        return username + "@Mail";  
    }  
}
```



Solução clássica: Herança

```
public class Notifier {  
    private final String username;  
    protected final DatabaseService databaseService;  
  
    public Notifier(String username) {  
        this.username = username;  
        databaseService = new DatabaseService();  
    }  
  
    public void send(String msg) {  
        String mail = databaseService.getMailFromUsername(username);  
        System.out.println("Sending " + msg + " by Mail to " + mail);  
    }  
  
    public String getUsername() {  
        return username;  
    }  
}
```



```
public class DatabaseService {  
    public String getMailFromUsername(String username) {  
        return username + "@Mail";  
    }  
}
```



Inheritance

Solução clássica: Herança

```
public class WhatsAppNotifier extends Notifier {  
    public WhatsAppNotifier(String username) {  
        super(username);  
    }  
  
    @Override  
    public void send(String msg) {  
        String phoneNbr = databaseService.getPhoneNbrFromUsername(getUsername());  
        System.out.println("Sending " + msg + " by WhatsApp on " + phoneNbr);  
    }  
}
```

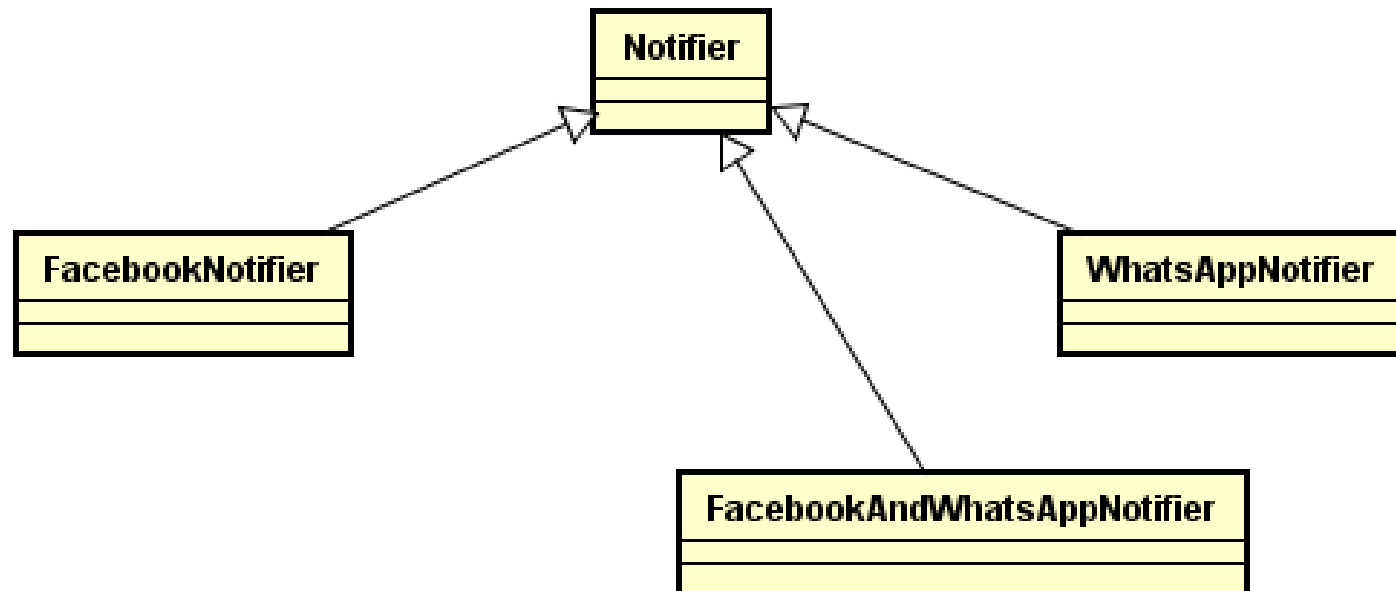
```
public class DatabaseService {  
    ...  
  
    public String getPhoneNbrFromUsername(String username) {  
        return username + "@Phone";  
    }  
  
    public String getFBNameFromUsername(String username) {  
        return username + "@Facebook";  
    }  
}
```

```
public class FacebookNotifier extends Notifier {  
    public FacebookNotifier(String username) {  
        super(username);  
    }  
  
    @Override  
    public void send(String msg) {  
        String fbName = databaseService.getFBNameFromUsername(getUsername());  
        System.out.println("Sending " + msg + " on Facebook to " + fbName);  
    }  
}
```



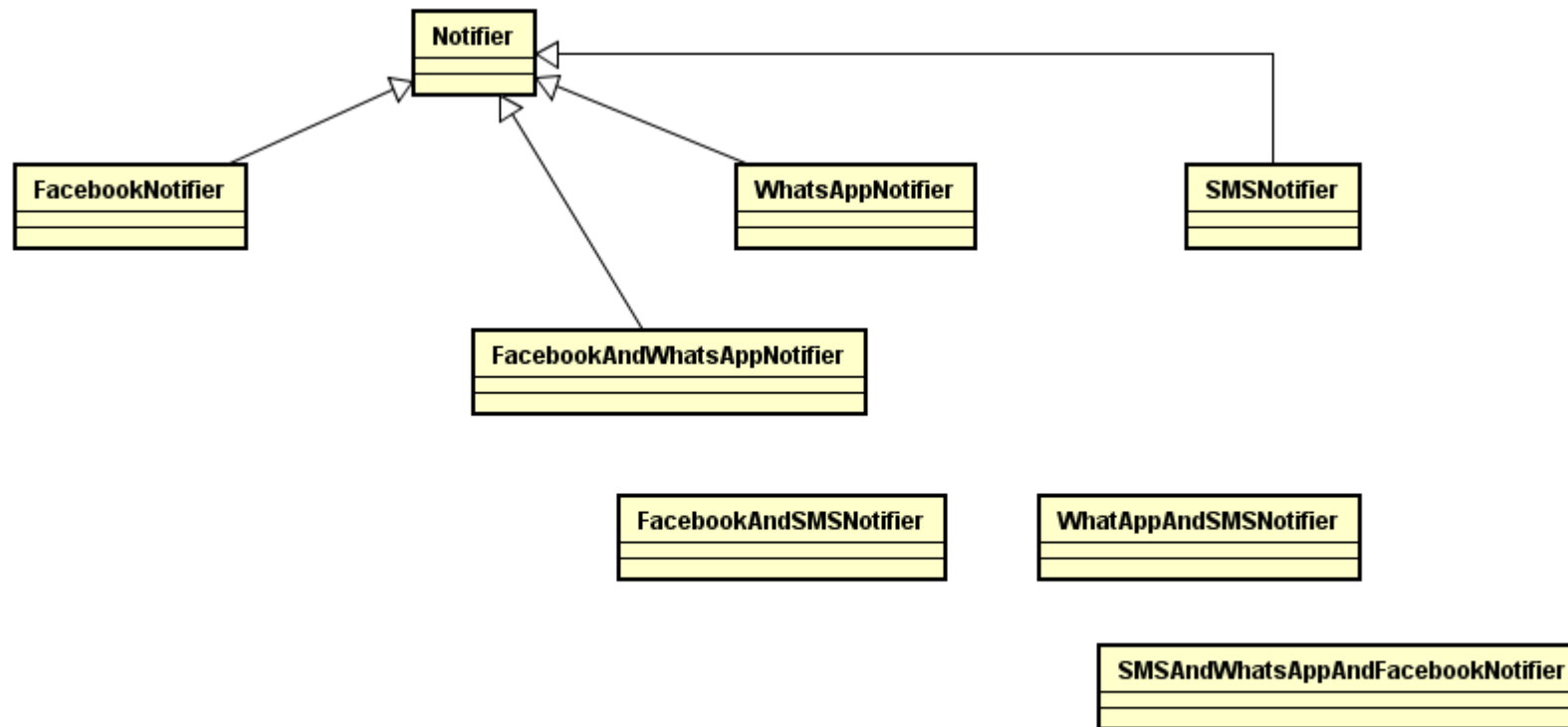
O Problema da solução com herança

- Precisa de muitas classes



O Problema da solução com herança

- Precisa de muitas classes



A SOLUÇÃO: Design Pattern Decorator

- É um padrão de projeto estrutural.
- Permite que novos comportamentos sejam anexados a um objeto colocando este objeto dentro de “embrulho especial” (special wrapper) que contém esses comportamentos.

Composição sim, herança não.

```
public class Notifier implements INotifier {
    private final String username;
    private final DatabaseService databaseService;

    public Notifier(String username) {
        this.username = username;
        databaseService = new DatabaseService();
    }

    @Override
    public void send(String msg) {
        String mail = databaseService.getMailFromUsername(username);
        System.out.println("Sending " + msg + " by Mail to " + mail);
    }

    @Override
    public String getUsername() {
        return username;
    }
}
```

```
public abstract class BaseNotifierDecorator
    implements INotifier {
    private final INotifier wrapped;
    protected final DatabaseService databaseService;

    BaseNotifierDecorator(INotifier wrapped) {
        this.wrapped = wrapped;
        databaseService = new DatabaseService();
    }

    @Override
    public void send(String msg) {
        wrapped.send(msg);
    }

    @Override
    public String getUsername() {
        return wrapped.getUsername();
    }
}
```

```
public class WhatsAppDecorator extends BaseNotifierDecorator {
    public WhatsAppDecorator(INotifier wrapped) {
        super(wrapped);
    }

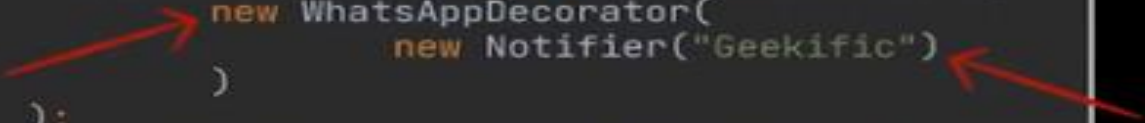
    @Override
    public void send(String msg) {
        super.send(msg);
        String phoneNbr = databaseService.getPhoneNbrFromUsername(getUsername());
        System.out.println("Sending " + msg + " by WhatsApp on " + phoneNbr);
    }
}
```

```
public interface INotifier {
    void send(String message);
    String getUsername();
}
```

~~Inheritance~~
Composition

Combinar funcionalidades

```
public static void main(String[] args) {  
    INotifier notifier = new FacebookDecorator(  
        new WhatsAppDecorator(  
            new Notifier("Geekific")  
        )  
    );  
    notifier.send("Like and Subscribe!!!");  
}
```



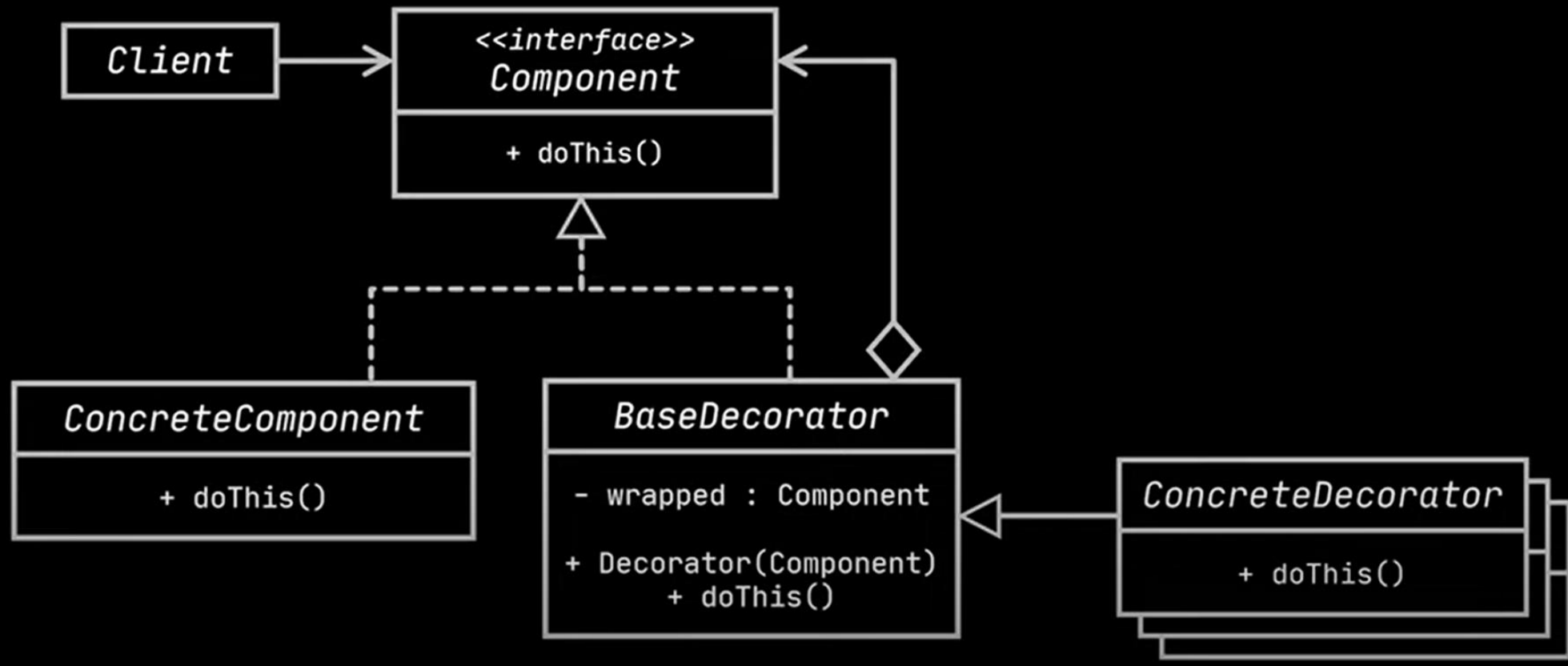
```
public class FacebookDecorator extends BaseNotifierDecorator {  
    ...  
  
    @Override  
    public void send(String msg) {  
        super.send(msg);  
        String fbName = databaseService.getUserNameFromUsername(getUsername());  
        System.out.println("Sending " + msg + " by Facebook to " + fbName);  
    }  
}
```

```
public class WhatsAppDecorator extends BaseNotifierDecorator {  
    ...  
  
    @Override  
    public void send(String msg) {  
        super.send(msg);  
        String phoneNbr = databaseService.getPhoneNbrFromUsername(getUsername());  
        System.out.println("Sending " + msg + " by WhatsApp on " + phoneNbr);  
    }  
}
```

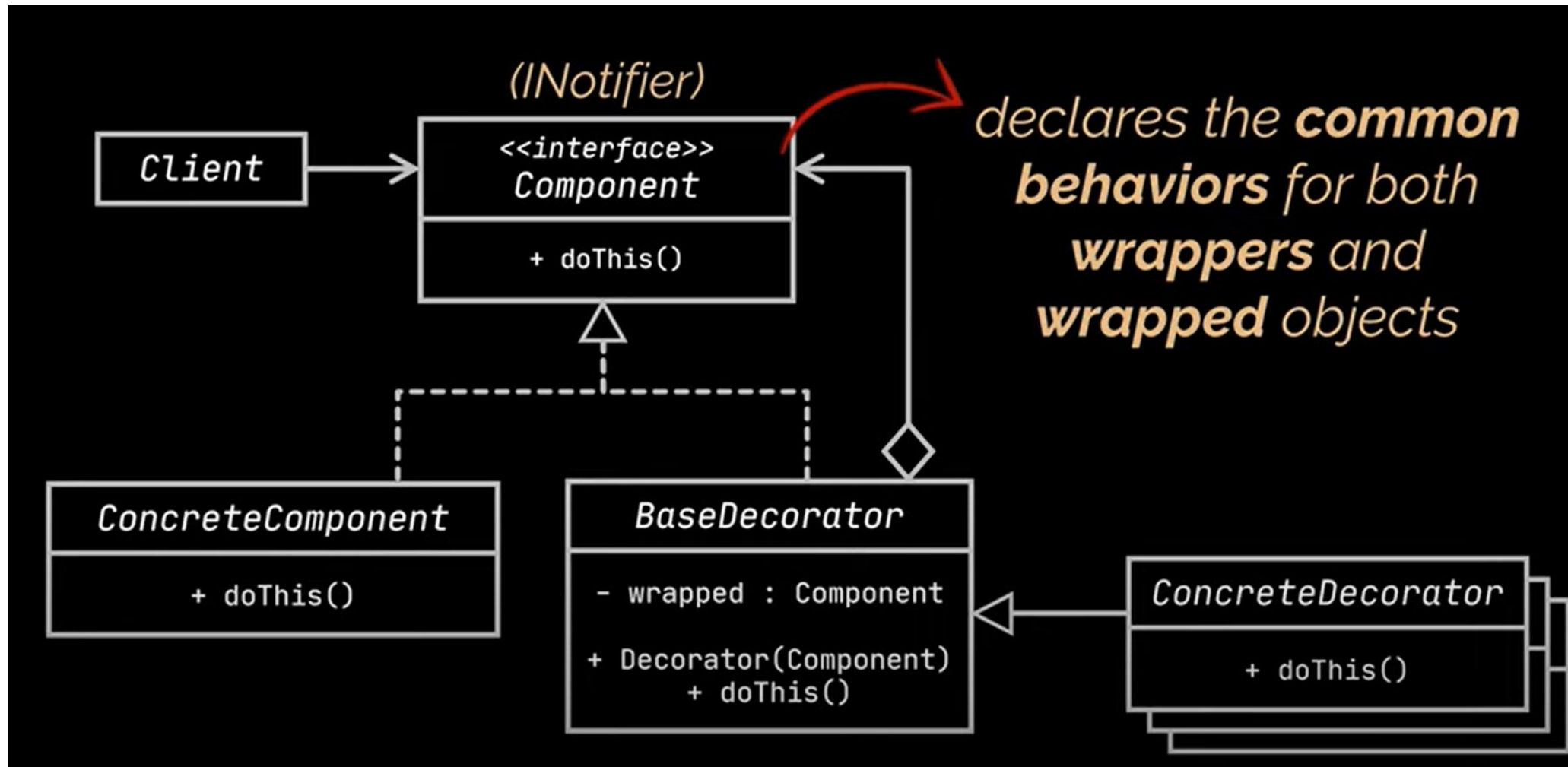
```
public class Notifier implements INotifier {  
    ...  
  
    @Override  
    public void send(String msg) {  
        String mail = databaseService.getMailFromUsername(getUsername());  
        System.out.println("Sending " + msg + " by Mail to " + mail);  
    }  
    ...  
}
```

Diagrama de Classes - Genérico

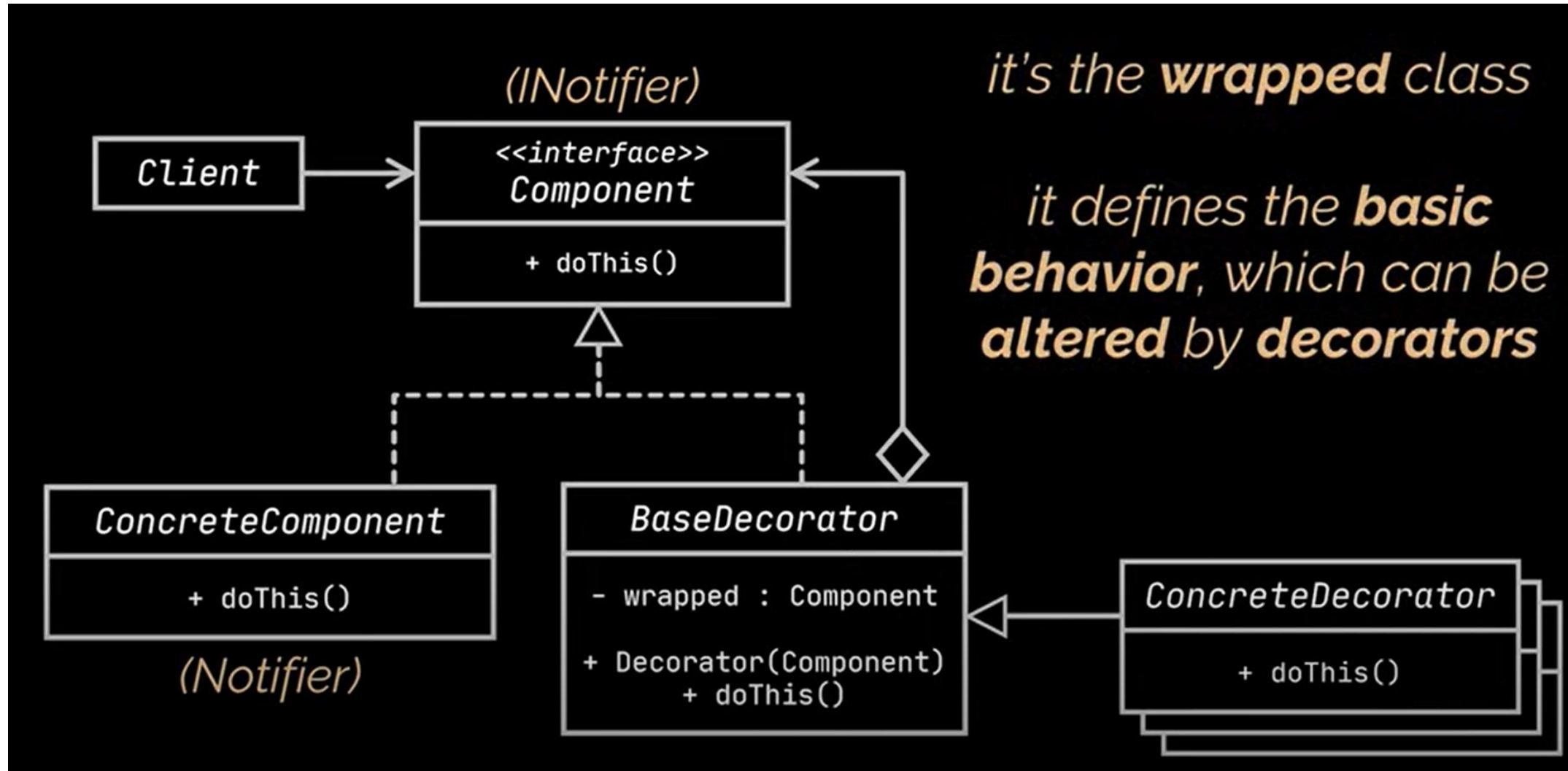
The Decorator Pattern Explained and Implemented in Java | Structural Design Patterns | Geekific



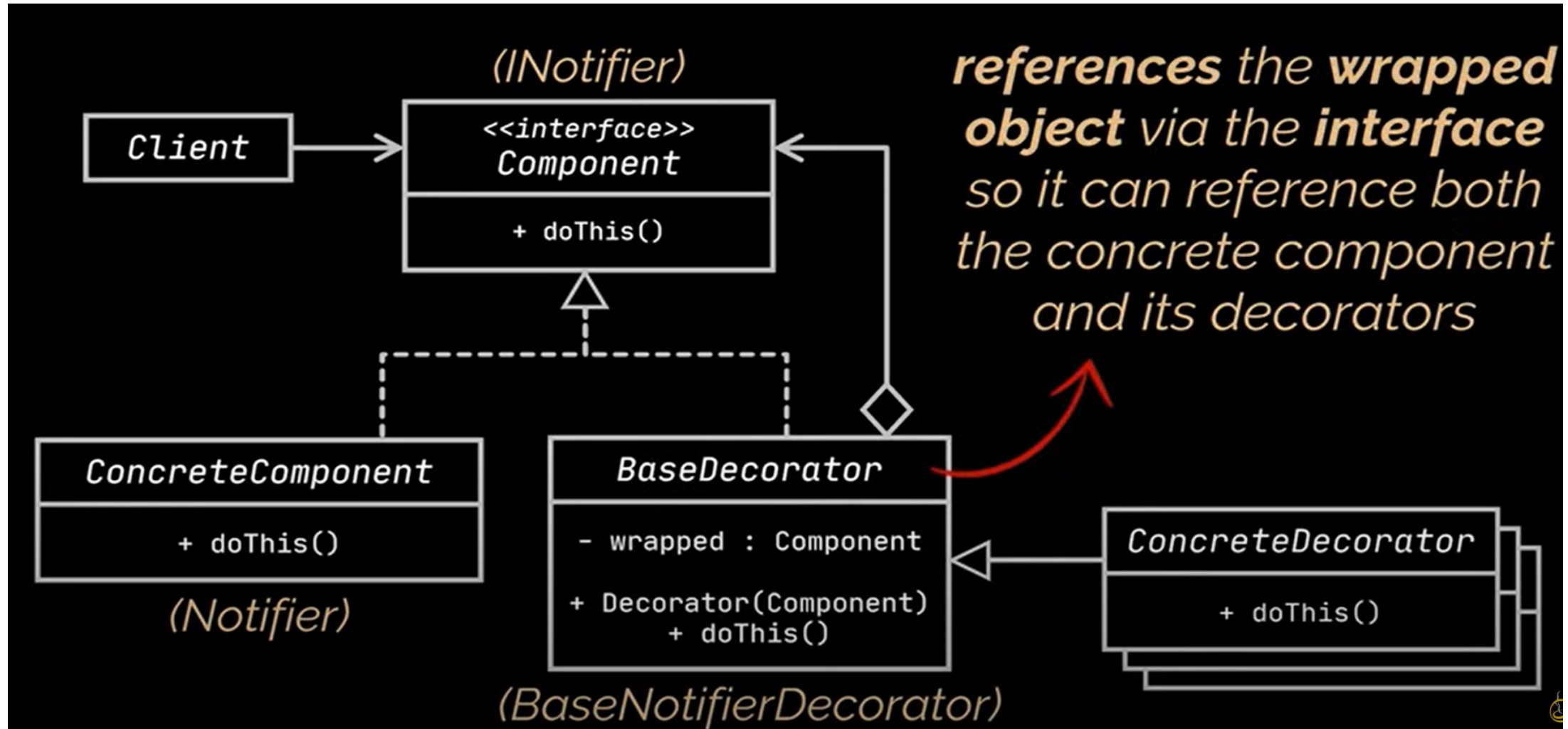
INotifier



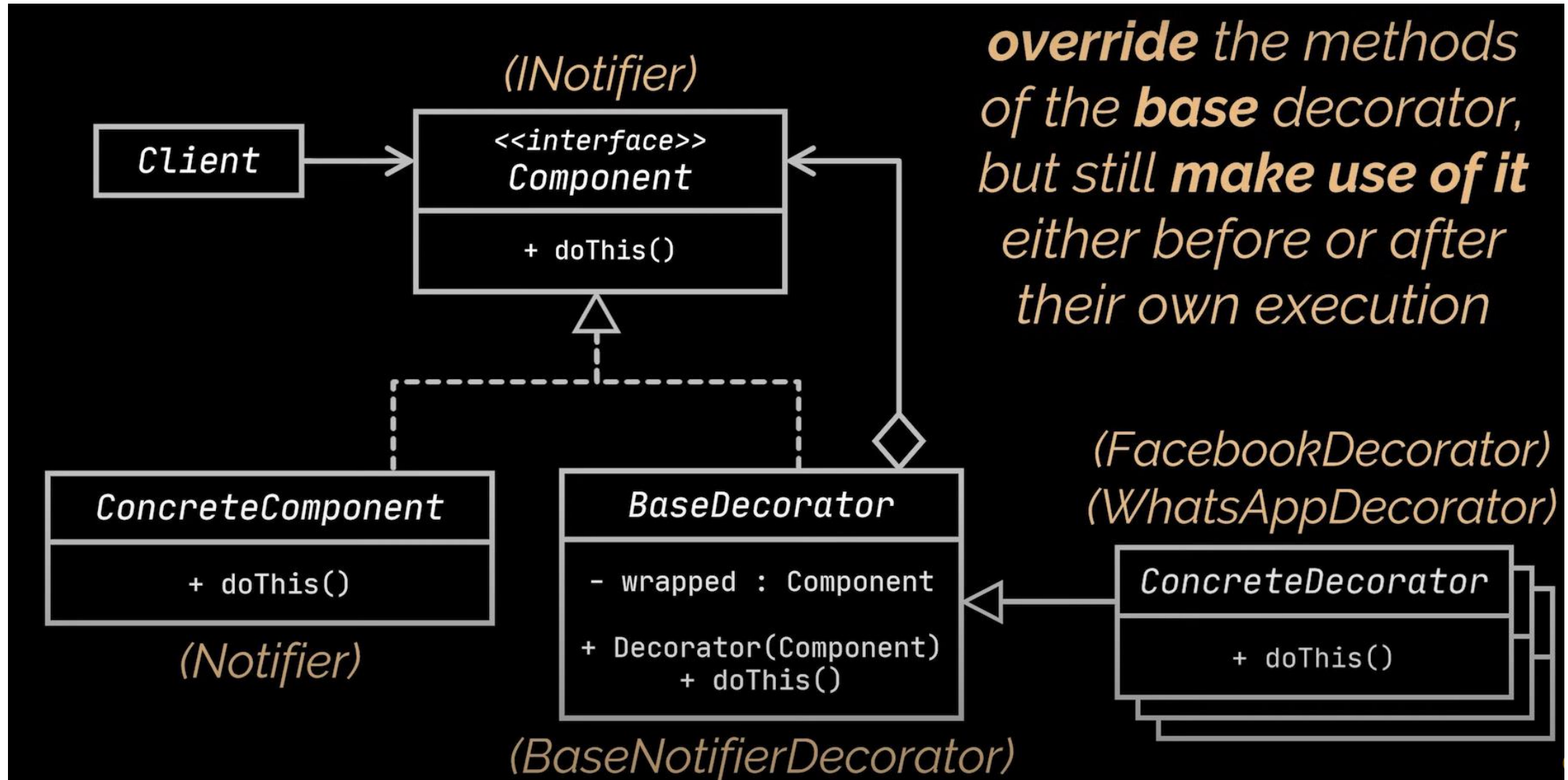
Notifier



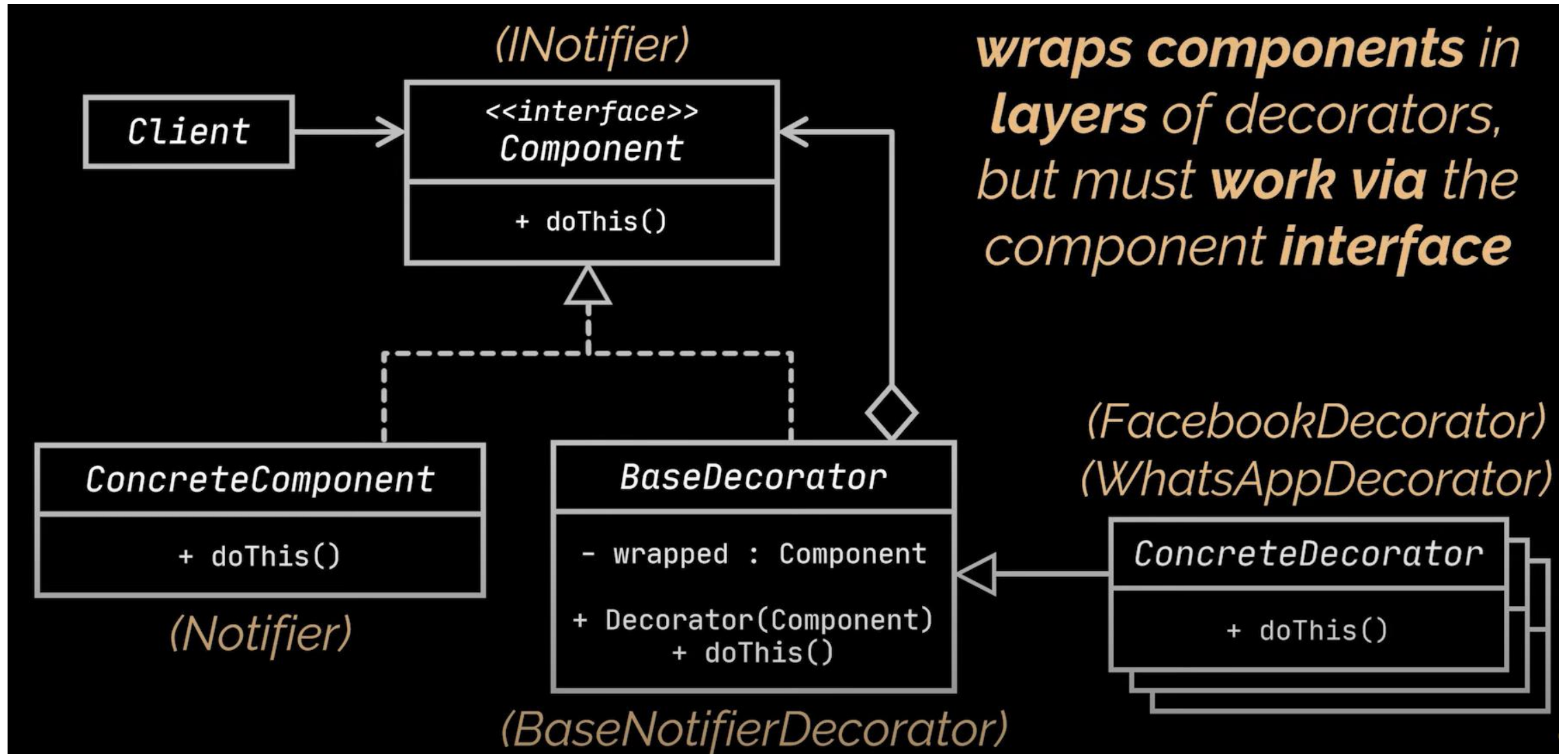
BaseNotifierDecorator



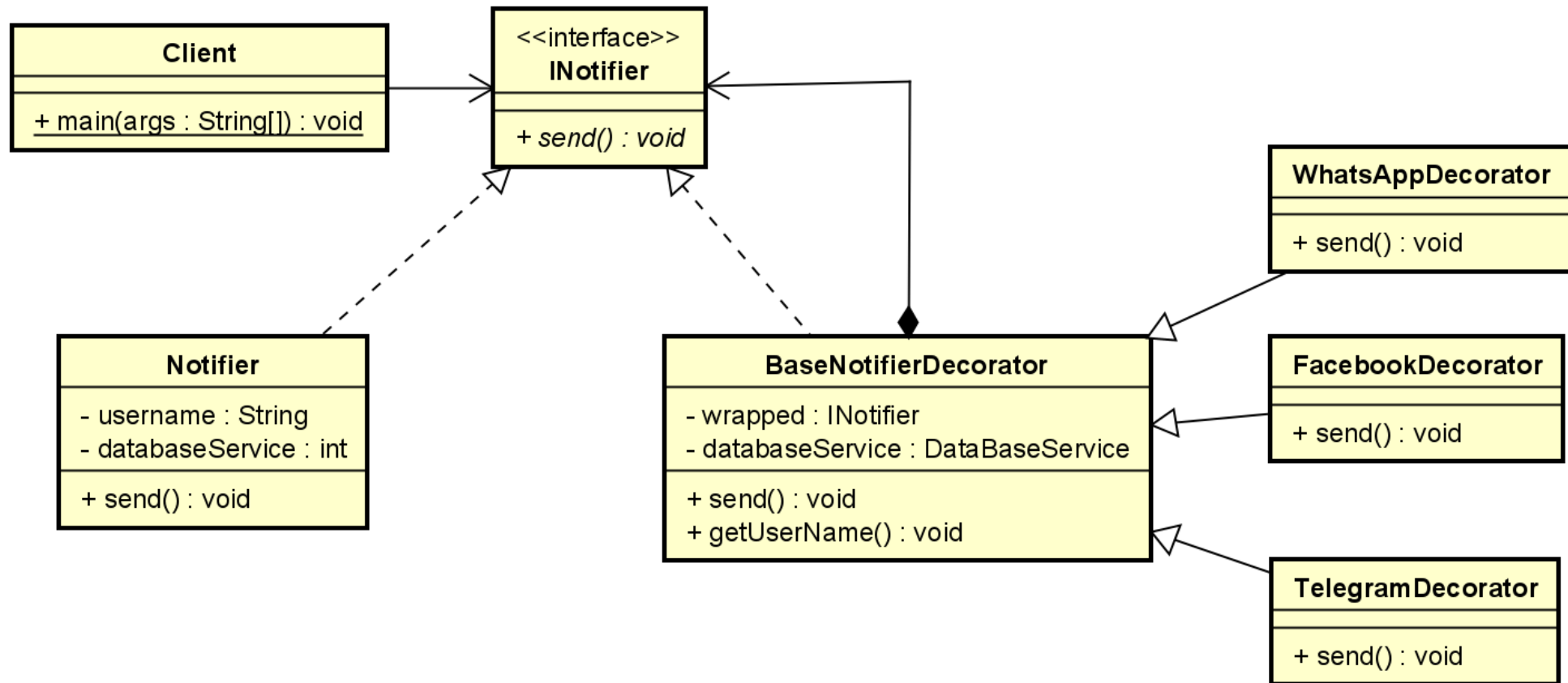
Classes concretas



A aplicação (Client)



A aplicação (Client)



Resumindo

